Assignment 2: Building a Structure from Motion Pipeline: Mapping and Localization

(80 marks + 10 marks bonus)

COL7680/JRL7680/COL780: Introduction to Computer Vision

Due: 15 March 2026, 11:59 PM

1 General Instructions

Academic Integrity. This assignment must be completed individually. Plagiarism or academic dishonesty will be addressed per institute policy and may result in severe penalties.
Evaluation. All submissions will be followed by a demo session in which students must explain their approach and answer questions about their implementation.
Discussion. Conceptual discussion and troubleshooting are encouraged via the course Piazza forum: `https://piazza.com/iitd.ac.in/spring2026/col7680`. Do not share code or solutions.
Submission Report. Include a report covering: implementation approach and design decisions, challenges encountered and how they were resolved, resources and references consulted, and any assumptions made.

2 Objective

Build a complete Structure from Motion (SfM) pipeline from video sequences to:

- Establish feature correspondences across frames.
- Estimate the Essential Matrix and recover relative camera motion via epipolar geometry.
- Triangulate initial 3D points and incrementally register new frames using PnP.
- Perform Bundle Adjustment to refine poses and 3D structure.
- Build a consistent sparse 3D map and localize unseen frames against it.

3 Dataset

Three rigid-object scenes are provided: **Truck**, **Barn**, and **Meeting Room**. Each scene is split into two parts:

- **Split A** — used for map construction (Tasks 1–5). Compute 2D feature observations, 3D points (2D–3D correspondences), and camera poses to form the reference map.
- **Split B** — used for localization (Task 6). Estimate poses for new sequences using the reference map from Split A. The map must remain fixed during localization.

The dataset (both splits and ground-truth `.ply` files) is available here.

4 Camera Calibration

Exact intrinsics are not provided. Initialize a pinhole camera model with:

$$x_0 = W/2, \quad y_0 = H/2, \quad f_x = f_y = 0.7W \tag{1}$$

where W and H are the image width and height in pixels. Intrinsics may optionally be refined during Bundle Adjustment.

5 Implementation Constraints

The following constraints apply across all tasks unless explicitly stated otherwise:

- **Task 1–3:** Use any OpenCV or NumPy functions to implement these tasks, but functions that solve the entire task in one step (e.g., `cv::sfm::reconstruct`) are not allowed.
- **Task 4:** Use scipy to implement the non-linear least square optimization for Bundle Adjustment task.
- **Feature extractor consistency for task 1–4:** The feature extractor and descriptor type chosen for Tasks 1–4 must remain fixed throughout the assignment and be reused identically during localization in Task 6. This choice must be documented in your report.
- **Task 5:** Use any pre-existing SfM system (e.g., COLMAP, OpenSfM) with the provided intrinsics. Use that system's own built-in feature extractor for all mapping and localization operations associated with this pipeline.
- **Task 6:** Reuse components from Tasks 1–4 in strict localization mode; no triangulation, map expansion, or Bundle Adjustment is permitted.
- Functions implemented in Task 2 (triangulation, reprojection error, nonlinear refinement) must be modularized for direct reuse in Tasks 3, 4, and 6.

6 Task 1: Feature Correspondence and Essential Matrix Estimation (10 Marks)

1. **Feature Detection and Matching.** Extract frames from Split A at regular intervals. Use any OpenCV method to detect keypoints, compute descriptors, and

match them between pairs of frames. Record your choice of feature extractor and descriptor type, as this must remain consistent in Task 6. Store the descriptors alongside their corresponding keypoints, as they will be associated with 3D map points in subsequent tasks.

2. **Essential Matrix Estimation.** Using the matched point correspondences, estimate the Essential Matrix E and enforce its rank-2 constraint via SVD.

   Implement a robust estimation strategy to handle outliers. Compare the Essential Matrices obtained (i) from all correspondences (E_1) and (ii) after robust estimation (E_2). Justify your analysis by plotting epipolar lines in Frame B for 10 matched points using both E_1 and E_2, and demonstrate which better satisfies the epipolar constraint.

7 Task 2: Triangulation and Pose Recovery (20 Marks)

1. **Pose Decomposition.** Decompose E via SVD to extract the four candidate (R, t) pairs.

2. **Linear Triangulation.** Implement linear triangulation to recover an initial 3D point X from matching correspondences across two views.

3. **Nonlinear Refinement.** Refine X by minimizing the total reprojection error over all observing views. For each view i, compute $y = M_i X$, project to image coordinates:

$$p'_i = \left(\frac{y_1}{y_3}, \frac{y_2}{y_3} \right) \tag{2}$$

   and minimize $e_i = p'_i - p_i$. Use the linear triangulation result as initialization and run for a fixed number of iterations (e.g., 10), without early stopping.

4. **Pose Disambiguation (Cheirality).** For each candidate (R, t), triangulate the inlier points and compute their depths Z_A and Z_B in camera frames A and B respectively. Select the (R, t) that maximizes the number of points satisfying $Z_A > 0$ and $Z_B > 0$.

5. **Deliverables.**

   For the initial 2-frame reconstruction of split A:

   (a) Report the four candidate (R, t) pairs obtained from Essential matrix decomposition and clearly justify the selected pose using the cheirality condition (number of points with positive depth in both views).

   (b) Report the reconstructed sparse 3D point cloud before and after non-linear refinement, including average reprojection errors and corresponding 3D plots.

6. **Note.** Students may refer to the book [1], Chapter 4, on Estimation, and to [2] for this task.

8 Task 3: Incremental Mapping and Multi-View Reconstruction (15 Marks)

1. **Initialization.** Fix the first camera as $P_A = (I, 0)$ and set $P_B = (R, t)$ from Task 2. Triangulate all inlier correspondences between Frame A and Frame B to obtain the initial sparse 3D map. Each 3D point must store the descriptors of its associated 2D observations for use during localization in Task 6.

2. **Incremental Pose Estimation.** For each subsequent frame, match features to the existing 3D map to establish 2D–3D correspondences. Estimate the camera pose via a robust PnP formulation, then refine it by minimizing reprojection error.

3. **Map Expansion.** Match features between the new frame and previously registered frames. Triangulate new 3D points, and add those with positive depth and acceptable reprojection error to the global map. Store the associated descriptors for each newly added 3D point.

4. **Deliverables**

   (a) Report the total number of registered cameras, reconstructed 3D points, and average reprojection error as the map incrementally grows.

   (b) Provide visualizations of the evolving sparse 3D map, including camera poses. Plots should clearly show the progressive addition of new points from the perspective of newly registered cameras/frames.

9 Task 4: Bundle Adjustment (10 Marks Bonus)

1. Jointly refine all camera poses $\{R_i, t_i\}$ and 3D points $\{X_j\}$ by minimizing the total reprojection error:

$$\min_{\{R_i, t_i, X_j\}} \sum_{i,j} \|p_{ij} - \pi(R_i, t_i, X_j)\|^2 \tag{3}$$

   Implement a nonlinear least-squares optimiser (e.g., using SciPy). Input your sparse 3D map and estimated camera poses from task 3. Bundle Adjustment refines only camera poses and 3D point coordinates — the descriptor-to-3D-point associations established during mapping must remain intact, as they are required for localization in Task 6.

2. **Deliverables.**

   Report the following before and after Bundle Adjustment:

   (a) **Mean Reprojection Error** over all observations.

   (b) **Chamfer Distance** between the reconstructed and ground-truth point clouds. Zero-center and normalize both clouds using the same scale before computing; do not apply any additional alignment.

   (c) **Qualitative Visualization** comparing the reconstructed and ground-truth models.

3. **Note.** The reconstructed map produced here is reused (fixed) in Task 6. Students may refer to the book [1], Chapter 18, on N-View Computational Methods, and to [2] for this task.

10 Task 5: Reconstruction Using a Pre-Existing SfM Pipeline (15 Marks)

1. Run an existing SfM system (e.g., COLMAP, OpenSfM) on all Split A sequences using the intrinsics from Section 4. Use the system's own built-in feature extractor for all processing. From the SfM output, extract:

   - Camera poses and sparse 3D points.
   - The descriptor associated with each 3D point (from its 2D observations), forming a 2D–3D lookup table.
   - A KD-Tree built over the stored descriptors (use approximate nearest-neighbor search, e.g., FLANN), where each entry links a descriptor to its corresponding 3D point. This structure is the map representation used for localization in Task 6.

2. **Deliverables.**

   Report for all three scenes using the same metrics as Task 4: Mean Reprojection Error, Chamfer Distance (same zero-mean normalization), and Qualitative Visualization.

   Comparison with Custom Pipeline (Tasks 1–4).

   (a) **Pose Comparison:** Compare estimated camera poses between the two pipelines for the same sequence.

   (b) **Reprojection Error:** Compare mean reprojection errors.

   (c) **Reconstruction Quality:** Compare sparse map density and Chamfer Distance.

11 Task 6: Camera Localization on Split B (20 Marks)

1. Using the fixed 3D maps from both the custom pipeline (Tasks 1–4) and the pre-existing SfM (Task 5), localize each frame in Split B independently. The 3D map must remain strictly fixed throughout — no triangulation, map expansion, or Bundle Adjustment is permitted.

   The localization procedure differs slightly between the two maps:

   - **Custom map:** Extract features from each Split B frame using the *same* feature extractor used in Tasks 1–4. Match descriptors against those stored in the custom map via nearest-neighbor search to establish 2D–3D correspondences. Estimate the camera pose via robust PnP and refine by minimizing reprojection error.

- **Pre-existing SfM map:** Extract features using the SfM system's own built-in feature extractor. Query the KD-Tree (built in Task 5) with the new frame's descriptors using approximate nearest-neighbor search (e.g., FLANN) to retrieve the closest stored descriptors and their associated 3D points, establishing 2D–3D correspondences. Estimate the camera pose via robust PnP and refine by minimizing reprojection error.

Note on the descriptor search: The KD-Tree is built over descriptors, not 3D point coordinates. The lookup proceeds as: new 2D descriptor $\rightarrow$ nearest stored descriptor $\rightarrow$ associated 3D point. This is a descriptor-space search, not a geometric one.

Localization failure: If the number of PnP inliers for a frame falls below a minimum threshold (e.g., 6 correspondences), mark that frame as a localization failure and exclude it from pose reporting. Document the failure rate per scene in your report.

2. **Deliverables**.

   All three evaluation metrics below must be computed and compared separately for both maps.

   (a) **Reprojection Error over the Sequence.** Report mean and standard deviation over all successfully localized frames, a temporal plot of reprojection error vs. frame index, and an analysis of stability and drift. Discuss whether error accumulates or remains stable.

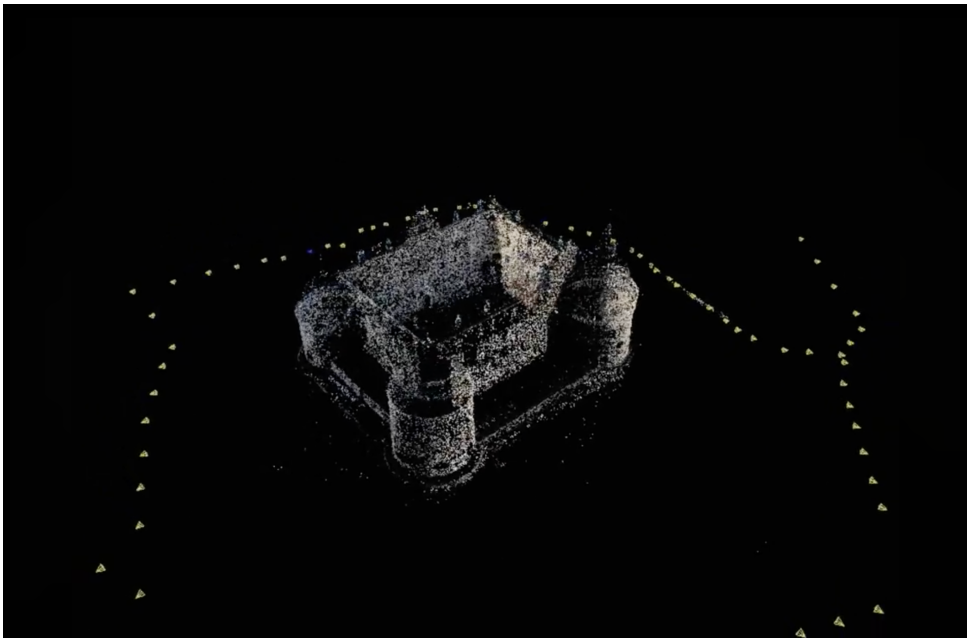

Figure 1: Camera trajectory visualization example. Camera poses are shown as frustums, with the apex at the camera center and the base representing the image plane. The forward-facing direction of each camera is indicated by the orientation of the frustum.

   (b) **Inlier Ratio Stability.** For each frame, record the number of inliers and the inlier ratio. Report the mean inlier ratio, a temporal plot, identification

of segments with weak geometric support, and the correlation between inlier ratio and reprojection error.

(c) **Camera Trajectory Visualization.** Plot the full 3D camera trajectories for both Split A (mapping) and Split B (localization) in the same coordinate frame using distinct colors. An example visualization is shown in Figure 1. Evaluate smoothness, consistency, drift, and observed failure cases.

12 Grading Summary

Task	**Marks**
Task 1: Feature Correspondence and Essential Matrix	10
Task 2: Triangulation and Pose Recovery	20
Task 3: Incremental Mapping	15
Task 4: Bundle Adjustment	10 (bonus)
Task 5: Pre-existing SfM Pipeline	15
Task 6: Localization	20
Total	**80 + 10 bonus**

Demo and Visualization Requirement. During the demo session, pipelines are expected to run in real time. Each task (excluding Bundle Adjustment, which is optional) must include a minimal live visualization of its progress — for example, matched features, epipolar lines, growing point cloud, or localized camera pose.

Permitted Libraries. Students are allowed to use any OpenCV or NumPy functions to implement the sfm pipeline for the tasks mentioned in the asssignment. However, using a single function that solves the entire assignment in one step (e.g., cv::sfm::reconstruct) is not permitted.

References

[1] R. I. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision.* Cambridge University Press, ISBN: 0521540518, second ed., 2004.

[2] MIT, "Nonlinear least squares notes." `https://vnav.mit.edu/material/17-18-NonLinearLeastSquares-notes.pdf`, n.d. Accessed: 2026-02-24.